Induzione di Alberi Decisionali per il Riconoscimento di Malattie Cardiovascolari

Università Politecnica delle Marche

Nicola Sebastianelli

Indice

1. Introduzione
2. Struttura del File CSV e Creazione del Database
3. Creazione del Database Elaborato
4. Creazione del Dataset per l'Induzione dell'Albero
5. Algoritmo di Induzione degli Alberi Decisionali
6. Matrice di Confusione e Valutazione del Modello
7. Risultati e Conclusioni

1. Introduzione

Le malattie cardiovascolari sono una delle principali cause di morte nel mondo, rendendo essenziale lo sviluppo di strumenti di supporto alla diagnosi precoce. In questo studio, abbiamo implementato e analizzato alberi decisionali per la classificazione della presenza di una malattia cardiovascolare, confrontando due criteri di selezione degli attributi:

- Indice di Gini
- Information Gain (IG)

L'implementazione è stata realizzata in Prolog, e il confronto tra i due metodi ha permesso di evidenziare differenze nella struttura dell'albero, nelle performance e nell'efficienza computazionale.

2. Struttura del File CSV e Creazione del Database

Il dataset originale è fornito in formato CSV ed è composto da 70.000 righe, ognuna delle quali rappresenta un paziente con le seguenti caratteristiche:

- Età (espressa in giorni, successivamente convertita in anni)
- Sesso (1: uomo, 2: donna)
- Altezza (in cm)
- Peso (in kg)
- Pressione arteriosa sistolica e diastolica
- Livelli di colesterolo e glucosio (1: normale, 2: sopra la norma, 3: molto sopra la norma)
- Abitudini di vita (fumo, alcol, attività fisica)
- Presenza di malattia cardiovascolare (0: assente, 1: presente)

Il file CSV è stato convertito in un database Prolog attraverso un predicato specifico:

```
:- use_module(library(csv)).

% Predicato principale per la conversione
convert_csv_to_prolog(InputCSV, OutputProlog) :-
    setup_call_cleanup(
        open(InputCSV, read, InStream),
        setup_call_cleanup(
            open(OutputProlog, write, OutStream),
```

```
        (
            read_csv(InStream, [_ | Rest]), % Ignora la prima riga
            write_facts(Rest, OutStream)
        ),
        close(OutStream)
    ),
    close(InStream)
).
```

3. Creazione del Database Elaborato

Una volta convertito il CSV in un database Prolog, è stato necessario elaborare i dati per migliorarne la qualità e la leggibilità. In particolare:

- L'età è stata convertita da giorni ad anni.
- Il BMI è stato calcolato e classificato in quattro categorie (sottopeso, normopeso, sovrappeso, obeso).
- Sono stati rimossi dati inconsistenti, come valori anomali della pressione sanguigna.

L'elaborazione è stata gestita tramite il seguente codice:

```
% Elaboro i valori individuali del fatto, aggiungendo il BMI e rimuovendo
altezza e peso
process_values([_ | [DaysStr, GenderStr, HeightStr, WeightStr,
SystolicBPStr, DiastolicBPStr | Rest]], [Years, Gender, BMIClass,
SystolicBP, DiastolicBP | Rest]) :-
    % Trasforma i giorni in anni arrotondati
    number_string(Days, DaysStr),
    YearsFloat is Days / 365.0,
    round_custom(YearsFloat, Years),
    classify_age_range(Years, AgeRange), % Classifica l'età in fasce

    % Escludi valori fuori dalle fasce (ad esempio, età 30)
    AgeRange \= 'invalid',

    % Converte Gender
    number_string(GenderNum, GenderStr),
    (   GenderNum =:= 1 -> Gender = 'female'
    ;   GenderNum =:= 2 -> Gender = 'male'),

    % Trasforma altezza e peso in numeri
    number_string(Height, HeightStr),
    number_string(Weight, WeightStr),

    % Converte pressione sanguigna in numeri
    number_string(SystolicBP, SystolicBPStr),
    number_string(DiastolicBP, DiastolicBPStr),

    % Controlla che altezza e peso siano validi
    valid_height_weight(Height, Weight),
```

```
    % Controlla che pressione sistolica e pressione diastolica siano
valide
    valid_blood_pressure(SystolicBP, DiastolicBP),

    % Calcola il BMI
    HeightMeters is Height / 100.0,
    BMI is Weight / (HeightMeters * HeightMeters),

    % Classifica il BMI
    classify_bmi(BMI, BMIClass).
```

Il risultato è stato salvato in database_cardio_elaborato.pl.

4. Creazione del Dataset per l'Induzione dell'Albero

Una volta elaborati i dati, è stato necessario creare un dataset strutturato per la costruzione degli alberi decisionali. Il file crea_dataset.pl ha generato i predicati necessari:

```
start :-
    tell('cardio_dataset.pl'),
    % Creazione dei predicati `a` per gli attributi
    setof(Age, aa(Age, _, _, _, _, _, _, _, _, _), AttributiAge),
    write('a(age,'), writeq(AttributiAge), writeln(').'),
    % Simile per gli altri attributi...
    generate_examples.
```

Successivamente, il dataset è stato suddiviso in training set (70%) e test set (30%) per garantire un'adeguata valutazione dei modelli:

```
% Predicato principale per dividere il dataset bilanciato
balanced_split_dataset(File, TrainingFile, TestFile, TrainRatio) :-
    consult(File),  % Carica il dataset
    % Separa i fatti in due gruppi
    findall(e(has_disease, Attributes), e(has_disease, Attributes),
HasDiseaseFacts),
    findall(e(no_disease, Attributes), e(no_disease, Attributes),
NoDiseaseFacts),
    % Determina il numero massimo di fatti bilanciati
    length(HasDiseaseFacts, HasDiseaseCount),
    length(NoDiseaseFacts, NoDiseaseCount),
    MinCount is min(HasDiseaseCount, NoDiseaseCount),
    % Prendi il numero bilanciato di fatti da ogni gruppo
    random_permutation(HasDiseaseFacts, ShuffledHasDisease),
    random_permutation(NoDiseaseFacts, ShuffledNoDisease),
    take(MinCount, ShuffledHasDisease, BalancedHasDisease),
    take(MinCount, ShuffledNoDisease, BalancedNoDisease),
    % Combina e mescola i fatti bilanciati
    append(BalancedHasDisease, BalancedNoDisease, AllBalancedFacts),
    random_permutation(AllBalancedFacts, ShuffledBalancedFacts),
    % Dividi in training e test set
    length(ShuffledBalancedFacts, Total),
    TrainCount is round(Total * TrainRatio),
```

```
    split_at(TrainCount, ShuffledBalancedFacts, TrainingSet, TestSet),
    % Trasforma i fatti con i predicati appropriati
    maplist(to_train_fact, TrainingSet, TrainingFacts),
    maplist(to_test_fact, TestSet, TestFacts),
    % Salva nei file
    save_facts(TrainingFile, TrainingFacts),
    save_facts(TestFile, TestFacts).

% Converte un fatto in un predicato train
to_train_fact(e(Class, Attributes), train(Class, Attributes)).

% Converte un fatto in un predicato test
to_test_fact(e(Class, Attributes), test(Class, Attributes)).

% Divide una lista in due parti alla posizione N
split_at(0, List, [], List).
split_at(N, [H|T], [H|Left], Right) :-
    N > 0,
    N1 is N - 1,
    split_at(N1, T, Left, Right).

% Prende i primi N elementi da una lista
take(0, _, []) :- !.
take(_, [], []) :- !.
take(N, [H|T], [H|R]) :-
    N > 0,
    N1 is N - 1,
    take(N1, T, R).

% Salva una lista di fatti in un file
save_facts(File, Facts) :-
    open(File, write, Stream),
    forall(member(Fact, Facts),
           (write(Stream, Fact), write(Stream, '.'), nl(Stream))),
    close(Stream).
```

5. Algoritmo di Induzione degli Alberi Decisionali

L'induzione degli alberi decisionali è stata implementata in Prolog, seguendo una logica ricorsiva. L'algoritmo procede nel seguente modo:

```
induce_albero(Albero) :-
    findall(e(Classe,Oggetto),train(Classe,Oggetto),Esempi),
    findall(Att,a(Att,_),Attributi),
    induce_albero(Attributi,Esempi,Albero),
    assert(alb(Albero)),
    salva_albero(Albero, 'albero_decisionale_IG.txt'). % Salva l'albero
nel file

induce_albero(_,[],null) :- !. %(1)
induce_albero(_,[e(Classe,_)|Esempi],l(Classe)) :- %(2)
    \+ (member(e(ClassX,_),Esempi), %non ci sono esempi
             ClassX \== Classe),!.         % di classi diverse
induce_albero(Attributi,Esempi,t(Attributo,SAlberi)) :-   %(3)
    sceglie_attributo_IG(Attributi,Esempi,Attributo), !, %scelta attributo
    del(Attributo,Attributi,Rimanenti),           % elimina Attributo
```

```
    a(Attributo,Valori),
    induce_alberi(Attributo,Valori,Rimanenti,Esempi,SAlberi).
induce_albero(_,Esempi,l(ClassePredominante)) :- %no attributi utili
    findall(Classe,member(e(Classe,_),Esempi),Classi),
    frequenza_maggiore(Classi, ClassePredominante).
```

L'attributo migliore viene scelto in base a due metriche diverse:

- **Indice di Gini**
  L'Indice di Gini misura l'impurità di un nodo e cerca di minimizzarla.
  Il coefficiente di Gini viene calcolato per ogni attributo e si sceglie quello con il valore più basso, ecco il codice:

```
sceglie_attributo(Attributi, Esempi, MigliorAttributo) :-
    setof(Dis/A,(member(A,Attributi),disuguaglianza(Esempi,A,Dis)),

[MinDis/MigliorAttributo|_]).

% disuguaglianza(Esempi, Attributo, Dis): Dis è la disuguaglianza
% combinata dei sottoinsiemi
%degli esempi partizionati dai valori dell'Attributo
disuguaglianza(Esempi,Attributo,Dis) :-
    a(Attributo,AttVals),
    somma_pesata(Esempi,Attributo,AttVals,0,Dis).

% somma_pesata(Esempi,Attributo,AttVals,SommaParziale,Somma): somma
% pesata delle disuguaglianze Gini = sum from{v} P(v) * sum from{i <> j}
% P(i|v)*P(j|v)
somma_pesata(_,_,[],Somma,Somma).
somma_pesata(Esempi,Att,[Val|Valori],SommaParziale,Somma) :-
    length(Esempi,N),             % quanti sono gli esempi

findall(C,(member(e(C,Desc),Esempi),soddisfa(Desc,[Att=Val])),EsClassi),
     %EsClassi = lista delle classi (con ripetizioni) di tutti gli esempi
per cui Att=Val
    length(EsClassi,NVal),NVal > 0, !,
    findall(P,(bagof(1,member(_,EsClassi),L),length(L,NVC),P is
NVC/NVal),ClDst),
    gini(ClDst,Gini),
    NuovaSommaParziale is SommaParziale + Gini*NVal/N,
    somma_pesata(Esempi,Att,Valori,NuovaSommaParziale,Somma);
    somma_pesata(Esempi,Att,Valori,SommaParziale,Somma). %nessun esempio
sodd

% gini(ListaProbabilità, IndiceGini): IndiceGini = SOMMATORIA Pi*Pj per
% tutti i,j tali per cui i\=j. E' equivalente a 1 - SOMMATORIA Pi*Pi su
% tutti gli i
gini(ListaProbabilità,Gini) :-
    somma_quadrati(ListaProbabilità,0,SommaQuadrati),
    Gini is 1-SommaQuadrati.
somma_quadrati([],S,S).
somma_quadrati([P|Ps],PartS,S) :-
```

```
    NewPartS is PartS + P*P,
    somma_quadrati(Ps,NewPartS,S).

% induce_alberi(Attributi, Valori, AttRimasti, Esempi, Salberi): induce
% decisioni SAlberi per sottoinsiemi di Esempi secondo i Valori degli
% Attributi
induce_alberi(_,[],_,_,[]).% nessun valore, nessun sottoalbero
induce_alberi(Att,[Val1|Valori],AttRimasti,Esempi,[Val1:Alb1|Alberi]) :-
    attval_subset(Att=Val1,Esempi,SottoinsiemeEsempi),
    induce_albero(AttRimasti,SottoinsiemeEsempi,Alb1),
    induce_alberi(Att,Valori,AttRimasti,Esempi,Alberi).

% attval_subset( Attributo = Valore, Esempi, Subset): Subset è il
% sottoinsieme di Examples che soddisfa la condizione Attributo = Valore
attval_subset(AttributoValore,Esempi,Sottoinsieme) :-

findall(e(C,O),(member(e(C,O),Esempi),soddisfa(O,[AttributoValore])),Sotto
insieme).

soddisfa(Oggetto,Descrizione) :-
    \+ (member(Att=Val,Descrizione),
        member(Att=ValX,Oggetto),
        ValX \== Val).

del(T,[T|C],C) :- !.
del(A,[T|C],[T|C1]) :-
      del(A,C,C1).
```

- **Information Gain**

  L'Information Gain misura la riduzione dell'entropia dopo la suddivisione dei dati. A differenza di Gini, Information Gain seleziona l'attributo con il valore massimo, ovvero quello che riduce maggiormente l'incertezza. Di seguito il codice associato:

```
% Sceglie l'attributo con Information Gain massimo
sceglie_attributo_IG(Attributi, Esempi, MigliorAttributo) :-
    setof(IG/A, (member(A, Attributi), guadagno_informazione(Esempi, A,
IG)),
          [_MaxIG/MigliorAttributo|_]).

% Calcola il guadagno di informazione
guadagno_informazione(Esempi, Attributo, IG) :-
    entropia(Esempi, H_S),
    a(Attributo, AttVals),
    somma_pesata_entropia(Esempi, Attributo, AttVals, 0, H_Sv),
    IG is H_S - H_Sv.

% Calcola l'entropia di un insieme di esempi
entropia(Esempi, Entropia) :-
    findall(Classe, member(e(Classe,_), Esempi), Classi),
    length(Esempi, N),
    findall(P, (bagof(1, member(C, Classi), L), length(L, N_C), P is
N_C/N), Probs),
```

```
    somma_entropia(Probs, 0, Entropia).

somma_entropia([], S, S).
somma_entropia([P|Ps], S0, S) :-
    (P =:= 0 -> S1 is S0; S1 is S0 - P * log(P) / log(2)),
    somma_entropia(Ps, S1, S).

% Somma pesata delle entropie
somma_pesata_entropia(_, _, [], Somma, Somma).
somma_pesata_entropia(Esempi, Att, [Val|Valori], SommaParziale, Somma) :-
    length(Esempi, N),
    findall(C, (member(e(C, Desc), Esempi), soddisfa(Desc, [Att=Val])),
EsempiSoddisfatti),
    length(EsempiSoddisfatti, NVal),
    (NVal > 0 ->
        entropia(EsempiSoddisfatti, H_Sv),
        NuovaSommaParziale is SommaParziale + (NVal/N) * H_Sv
    ;
        NuovaSommaParziale is SommaParziale
    ),
    somma_pesata_entropia(Esempi, Att, Valori, NuovaSommaParziale, Somma).
```

6. Matrice di Confusione e Valutazione del Modello

Per valutare le prestazioni dell'albero decisionale, è stata utilizzata la matrice di confusione, che permette di analizzare la distribuzione delle predizioni corrette e degli errori:

- Vero Positivo (VP): Il modello predice correttamente la presenza della malattia.
- Falso Positivo (FP): Il modello predice la malattia, ma il paziente non è malato.
- Vero Negativo (VN): Il modello predice correttamente l'assenza della malattia.
- Falso Negativo (FN): Il modello predice assenza della malattia, ma il paziente è malato.

Il codice Prolog per la generazione della matrice di confusione è il seguente:

```
% Matrice di confusione e metriche
stampa_matrice_di_confusione :-
    alb(Albero),
    findall(Classe/Oggetto, test(Classe, Oggetto), TestSet),
    length(TestSet, N),
    valuta(Albero, TestSet, VN, 0, VP, 0, FN, 0, FP, 0, NC, 0),
    A is (VP + VN) / (N - NC), % Accuratezza
    E is 1 - A,                % Errore
    writeln('===== Matrice di Confusione ====='),
    write('Test effettuati: '), writeln(N),
    write('Test non classificati: '), writeln(NC),
    write('Veri Negativi: '), write(VN), write('   Falsi Positivi: '),
writeln(FP),
```

```
    write('Falsi Negativi: '), write(FN), write('   Veri Positivi: '),
writeln(VP),
    write('Accuratezza: '), writeln(A),
    write('Errore: '), writeln(E).

% Valutazione del test set
valuta(_,[],VN,VN,VP,VP,FN,FN,FP,FP,NC,NC).

valuta(Albero,[no_disease/Oggetto|Coda],VN,VNA,VP,VPA,FN,FNA,FP,FPA,NC,NCA
) :-
    classifica(Oggetto,no_disease,Albero), !,
    VNA1 is VNA + 1,
    valuta(Albero,Coda,VN,VNA1,VP,VPA,FN,FNA,FP,FPA,NC,NCA).

valuta(Albero,[has_disease/Oggetto|Coda],VN,VNA,VP,VPA,FN,FNA,FP,FPA,NC,NC
A) :-
    classifica(Oggetto,has_disease,Albero), !,
    VPA1 is VPA + 1,
    valuta(Albero,Coda,VN,VNA,VP,VPA1,FN,FNA,FP,FPA,NC,NCA).

valuta(Albero,[has_disease/Oggetto|Coda],VN,VNA,VP,VPA,FN,FNA,FP,FPA,NC,NC
A) :-
    classifica(Oggetto,no_disease,Albero), !,
    FNA1 is FNA + 1,
    valuta(Albero,Coda,VN,VNA,VP,VPA,FN,FNA1,FP,FPA,NC,NCA).

valuta(Albero,[no_disease/Oggetto|Coda],VN,VNA,VP,VPA,FN,FNA,FP,FPA,NC,NCA
) :-
    classifica(Oggetto,has_disease,Albero), !,
    FPA1 is FPA + 1,
    valuta(Albero,Coda,VN,VNA,VP,VPA,FN,FNA,FP,FPA1,NC,NCA).

valuta(Albero,[_/Oggetto|Coda],VN,VNA,VP,VPA,FN,FNA,FP,FPA,NC,NCA) :-
    classifica(Oggetto,nc,Albero), !,
    NCA1 is NCA + 1,
    valuta(Albero,Coda,VN,VNA,VP,VPA,FN,FNA,FP,FPA,NC,NCA1).

% Classificazione degli oggetti
classifica(Oggetto,nc,t(Att,Valori)) :-
    member(Att=Val,Oggetto),
    member(Val:null,Valori).

classifica(Oggetto,Classe,t(Att,Valori)) :-
    member(Att=Val,Oggetto),
    member(Val:l(Classe),Valori).

classifica(Oggetto,Classe,t(Att,Valori)) :-
    member(Att=Val,Oggetto),
    delete(Oggetto,Att=Val,Resto),
    member(Val:t(AttFiglio,ValoriFiglio),Valori),
    classifica(Resto,Classe,t(AttFiglio,ValoriFiglio)).
```

7. Risultati e Conclusioni

L’analisi ha mostrato che:

- L’albero basato su Gini tende a essere più profondo, con più nodi intermedi e minore ampiezza.
- L’albero basato su Information Gain è più largo e meno profondo, con un numero maggiore di ramificazioni iniziali.
- L’accuratezza ottenuta è stata del 75% per entrambi i modelli, indicando che la qualità dei dati incide più della scelta del criterio di suddivisione.

Possibili miglioramenti futuri includono:

- Aggiunta di nuovi attributi (es. dati genetici o anamnesi familiare).
- Uso di modelli come Random Forest per migliorare la robustezza delle predizioni.
- Ottimizzazione dell’algoritmo di induzione per ridurre il tempo di computazione.